

Comparação Entre Algoritmos de Escalonamento

Breno C. R. Nakamura¹, Estevan T. Santos¹, Gabriel L. Bonavina¹,
Jonas L. Durão¹, Natália V. A. do Val¹, Vinicius A. L. Andrade¹

¹Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (UNIFESP)
Av. Cesare Monsueto Giulio Lattes, 1201 – 12247-014 – São José dos Campos, SP

Abstract. *This project presents a practical application of Operating Systems concepts through the exploration and modification of the MINIX 3.4.0rc6 microkernel architecture. The work details everything from the environment setup process to direct source code alterations for implementing three scheduling algorithms: First-Come, First-Served (FCFS), Round-Robin (RR), and Multiple Queues with Exponential Quantum (MF). Compared to the default scheduler under CPU-bound and I/O-bound workloads, the results demonstrate how context switch overhead and hardware bottlenecks directly affect the efficiency of each algorithm.*

Resumo. *Este projeto apresenta uma aplicação prática dos conceitos de Sistemas Operacionais por meio da exploração e modificação da arquitetura de microkernel do MINIX 3.4.0rc6. O trabalho descreve desde o processo de configuração do ambiente até a alteração direta do código-fonte para a implementação de três algoritmos de escalonamento: First-Come, First-Served (FCFS), Round-Robin (RR) e Múltiplas Filas com Quantum Exponencial (MF). Comparados ao escalonador padrão sob cargas CPU-bound e I/O-bound, os resultados demonstram como o overhead de troca de contexto e os gargalos de hardware afetam diretamente a eficiência de cada algoritmo.*

O código-fonte completo deste projeto pode ser encontrado no repositório do Github: <https://github.com/brenonak/minix>

1. Introdução

O MINIX 3 é um sistema operacional de código aberto criado por Andrew S. Tanenbaum. Ele foi projetado para ser confiável e flexível, sendo utilizado para fins didáticos [Tanenbaum and Woodhull 2008]. Sua arquitetura é baseada em microkernel, na qual apenas as funções mais básicas, como o gerenciamento de interrupções e comunicação entre processos, são executadas no núcleo do sistema. Os demais componentes, como drivers, gerenciamento de arquivos e o escalonador, operam separados do núcleo, como processos independentes no espaço do usuário. Essa separação ajuda a evitar que falhas em um serviço comprometam o sistema inteiro, além de facilitar o estudo e modificação de seu código-fonte.

Este projeto utiliza a versão MINIX 3.4.0rc6 com o objetivo de aplicar na prática os conceitos teóricos da disciplina de Sistemas Operacionais. O trabalho explora o funcionamento interno do sistema por meio de sua instalação em ambiente virtualizado, alteração de chamadas no kernel e, principalmente, através da implementação e análise comparativa de desempenho de diferentes algoritmos de escalonamento de processos sob variadas cargas de trabalho.

2. Instalação

A primeira etapa deste projeto consistiu na instalação e configuração inicial do MINIX na máquina virtual. Para isso, foi necessário seguir o manual de instruções apresentado no enunciado. Imagens da instalação podem ser encontradas nas Figuras 1 e 2.

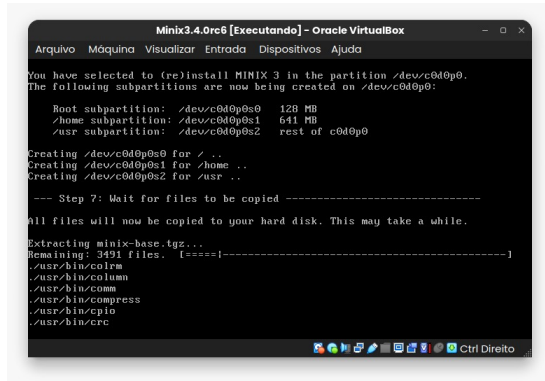


Figura 1. Etapa 7 de instalação do MINIX.

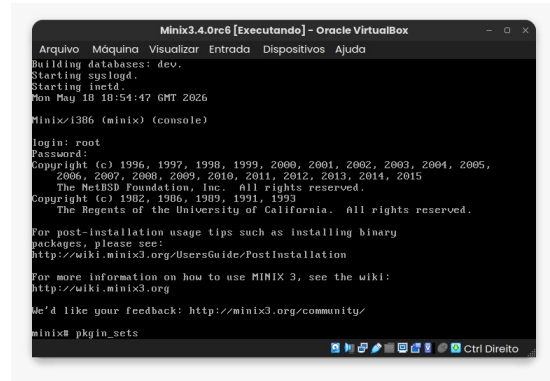


Figura 2. Atualização dos pacotes do MINIX.

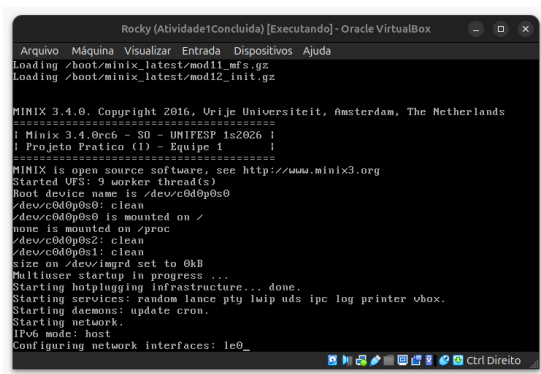
Além da instalação do MINIX, também foi necessário fazer a configuração de rede no *Virtual Box*, ajustando a política de endereçamento MAC para incluir apenas os endereços MAC de placas em rede NAT.

Além disso, foi necessário fazer o *clone* do repositório no ambiente virtual, para realizar as modificações que serão detalhadas em seções futuras deste relatório. Para isso, foi necessário configurar um Token de Acesso Pessoal do Github, o que nos permitiu executar os comandos do *git* no ambiente virtual. Encontramos dificuldades nessa parte, já que no ambiente virtual não é possível utilizar os comandos de cópia e colagem. Para contornar esse problema, criamos um arquivo `clonar.sh` e um arquivo `meu.token.txt`, que contém o token de acesso pessoal do Github, e, ao rodar `clonar.sh`, o repositório é clonado no ambiente virtual. Optamos por esta abordagem pois é uma forma de garantir que o token do Github esteja correto no momento da execução do script.

Por fim, faz-se necessário ressaltar as configurações de hardware da máquina virtual. Para a execução deste projeto utilizamos uma máquina virtual com 1 *core* de processador, 1GB de memória RAM e 4GB de HD dinamicamente alocado.

3. Mudança nos Banners

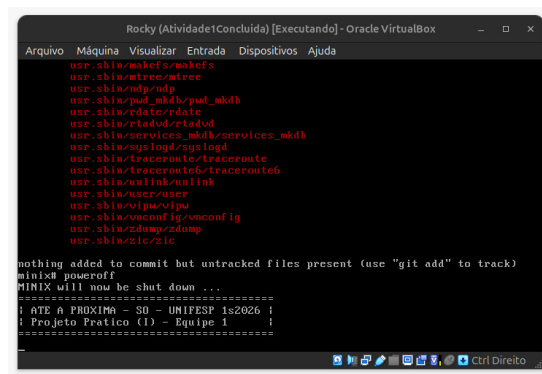
Para a mudança no banner durante *boot* e *poweroff*, foi necessário navegar no diretório `minix/minix/kernel/main.c`. Na função `announce()`, foram acrescentados *prints* para reproduzir o banner indicado no enunciado. Além disso, na função `prepare_shutdown()`, também inserimos *prints* para a impressão do banner. Os resultados podem ser encontrados nas Figuras 3 e 4.



```
Rocky (Atividade1Concluida) [Executando] - Oracle VirtualBox
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda
Loading /boot/minix_latest/modlib_mfs.gz
Loading /boot/minix_latest/modlib_init.gz

MINIX 3.4.0. Copyright 2016, Urije Universiteit, Amsterdam, The Netherlands
=====
! Minix 3.4.0rc6 - SO - UNIFESP 1s2026 !
! Projeto Pratico (I) - Equipe 1 !
=====
MINIX is open source software, see http://www.minix3.org
Started VFS: 9 worker thread(s)
Root device name is /dev/c0d0p0s0
/dev/c0d0p0s0: clean
/dev/c0d0p0s0 is mounted on /
none is mounted on /proc
/dev/c0d0p0s2: clean
/dev/c0d0p0s1: clean
Size on /dev/imgrd set to 0kB
Multiuser startup in progress ...
Starting hotplugging infrastructure... done.
Starting services: random lance pty lwp uds ipc log printer vbox.
Starting daemons: update cron.
Starting network.
IPV6 mode: host
Configuring network interfaces: le0_
```

Figura 3. Resultado do banner de boot.

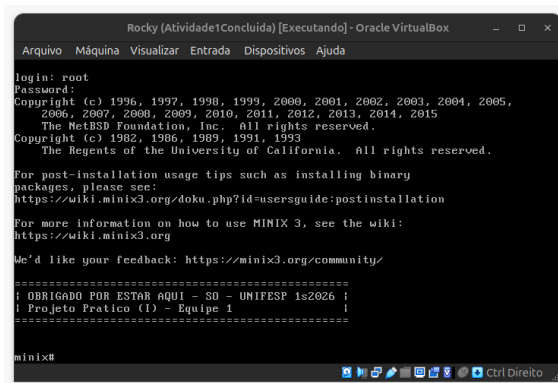


```
Rocky (Atividade1Concluida) [Executando] - Oracle VirtualBox
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda
usr/sbin/makefs/makefs
usr/sbin/mtrcc/mtrcc
usr/sbin/ndp/ndp
usr/sbin/pad_mkdb/pad_mkdb
usr/sbin/rdate/rdate
usr/sbin/rtdad/rtdad
usr/sbin/services_mkdb/services_mkdb
usr/sbin/syslogd/syslogd
usr/sbin/traceroute/traceroute
usr/sbin/traceroute6/traceroute6
usr/sbin/unlink/unlink
usr/sbin/user/user
usr/sbin/vipw/vipw
usr/sbin/vnconf/vnconf
usr/sbin/zdump/zdump
usr/sbin/zic/zic

nothing added to commit but untracked files present (use "git add" to track)
minix poweroff
minix will now be shut down ...
=====
! ATE A PROXIMA - SO - UNIFESP 1s2026 !
! Projeto Pratico (I) - Equipe 1 !
=====
```

Figura 4. Resultado do banner de poweroff.

Por fim, para a modificação do banner de *login*, foi necessário navegar pelos diretórios `minix/etc/motd`, onde inserimos o banner indicado pelo enunciado. O resultado da modificação pode ser encontrado na Figura 5.



```
Rocky (Atividade1Concluida) [Executando] - Oracle VirtualBox
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda
login: root
Password:
Copyright (c) 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005,
2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015
The BSD Foundation, Inc. All rights reserved.
Copyright (c) 1982, 1986, 1989, 1991, 1993
The Regents of the University of California. All rights reserved.

For post-installation usage tips such as installing binary
packages, please see:
https://wiki.minix3.org/doku.php?id=usersguide:postinstallation

For more information on how to use MINIX 3, see the wiki:
https://wiki.minix3.org

We'd like your feedback: https://minix3.org/community/

=====
! OBRIGADO POR ESTAR AQUI - SO - UNIFESP 1s2026 !
! Projeto Pratico (I) - Equipe 1 !
=====
minix#
```

Figura 5. Resultado do banner de login.

4. Mudança `exec.c`

Para registrar a execução de processos exibindo o comando e seu caminho absoluto, a implementação foi dividida entre o *Virtual File System* (VFS) e o *Process Manager* (PM), respeitando a modularidade da arquitetura do MINIX [MINIX 3 Project].

Inicialmente, adicionou-se o campo `execpath` à estrutura compartilhada `exec_info` (no arquivo `libexec.h`) como um campo de comunicação entre o VFS e o PM. No VFS (`vfs/exec.c`), logo após o carregamento do executável, o caminho absoluto armazenado na variável `finalexec` é copiado para `execi.args.execpath` utilizando a função `strncpy`. Em seguida, o VFS aciona o PM por meio de `libexec_pm_newexec`.

A exibição da mensagem foi centralizada no PM (`pm/exec.c`). Na rotina `do_newexec`, os dados preenchidos pelo VFS são recebidos via `sys_datacopy`. Após a cópia dos argumentos para a memória do PM, inseriu-se um `printf` para exibir a mensagem Executando: `<caminho/comando>`. O resultado dessa modificação pode ser encontrado na Figura 6.

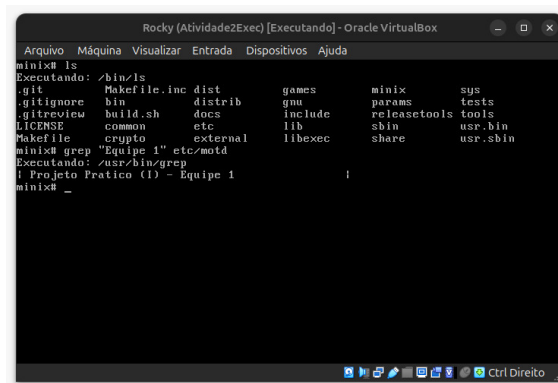


Figura 6. Resultado da modificação do `exec.c`.

Por fim, é válido destacar que todas os caminhos de arquivos modificados durante esta fase do projeto foram encontrados por meio da documentação oficial do MINIX [MINIX 3 Project].

5. Descrição do Exercício Proposto

Este projeto avalia o desempenho de diferentes algoritmos de escalonamento no MINIX 3.4.0rc6. Além do algoritmo padrão, o código-fonte do sistema foi modificado para implementar três escalonadores diferentes: *Round-Robin*, Múltiplas Filas com *Quantum* Exponencial e *First-Come, First-Served*. A eficiência de cada escalonador foi testada sob perfis *CPU-bound*, com cargas de 1 milhão de operações por processo, e *I/O-bound*, com 10 mil operações. A avaliação desenvolvida utilizou o programa `teste_processos.c`, submetendo o sistema a escalas de 10, 50, 100 e 200 processos simultâneos.

6. Descrição dos Algoritmos de Escalonamento

Para fundamentar as políticas implementadas neste projeto, é essencial estabelecer a base teórica do escalonamento de processos. Em sistemas multiprogramados, o escalonador é o componente do sistema operacional responsável por decidir qual tarefa terá acesso à CPU. Os algoritmos que guiam essa escolha dividem-se em duas categorias: os não preemptivos, nos quais a tarefa executa ininterruptamente até ser bloqueada ou liberar o processador de forma voluntária; e os preemptivos, que forçam a interrupção do processo após um intervalo de tempo máximo predefinido, conhecido como *quantum* [Tanenbaum 2016, p. 103–106]. Os algoritmos abordados a seguir englobam ambas as categorias: o modelo padrão do MINIX, o *Round-Robin* e as Múltiplas Filas com *Quantum* Exponencial operam de forma preemptiva, enquanto o FCFS adota o modelo não preemptivo.

6.1. Algoritmo Padrão do MINIX

O algoritmo de escalonamento padrão do MINIX 3 utiliza um escalonador preemptivo baseado em múltiplas filas de prioridade com ajuste dinâmico. O sistema gerencia 16 filas (0 a 15), nas quais o nível 0 possui a maior prioridade e o nível 15, a menor. O escalonador sempre seleciona o processo localizado no topo da fila de maior prioridade que não esteja vazia. As filas superiores, de 0 a 6, são destinadas a processos críticos do núcleo e a servidores essenciais do sistema. Já os processos de usuário são alocados entre as filas 7 (`USER_Q`) e 14 (`MIN_USER_Q`). A fila 15 (`IDLE_Q`), de menor prioridade, é

reservada ao processo ocioso do sistema, executado apenas quando não há nenhuma outra instrução pendente no processador.

Devido à arquitetura de microkernel do MINIX 3, o escalonamento ocorre tanto no espaço de núcleo quanto no espaço de usuário. No espaço de núcleo, implementado principalmente em `/minix/kernel/proc.c`, encontram-se os mecanismos de baixo nível, fornecendo as operações `enqueue()` e `dequeue()`, responsáveis pela manipulação das estruturas de dados, além da função `pick_proc()`, responsável por selecionar o próximo processo a receber a CPU com base na prioridade estrita.

Por outro lado, a política de escalonamento é implementada no espaço de usuário no diretório `/minix/servers/sched/`, com destaque para o arquivo `schedule.c`, onde o servidor de escalonamento `sched` gerencia o tempo de execução dos processos. Internamente, cada fila utiliza o algoritmo Round-Robin, no qual cada processo recebe uma fatia limitada de tempo de CPU (*quantum*). O servidor `sched` reduz a prioridade de processos pesados (*CPU-bound*) que esgotam seu *quantum* e promove processos interativos (*I/O-bound*) por meio do mecanismo de *aging*, implementado na função `balance_queues()`, garantindo maior responsividade e evitando inanição.

6.2. Round-Robin (RR)

O algoritmo de escalonamento por chaveamento circular (ou *Round-Robin*) é um dos métodos mais simples e tradicionais devido à sua fácil implementação. Seu funcionamento baseia-se na definição de uma fatia de tempo de CPU (*quantum*). O processo em execução retém a CPU por esse tempo limite; ao esgotá-lo, sofre preempção e retorna ao final da fila, cedendo espaço para o próximo processo [Tanenbaum 2016, p. 109–110].

Para implementar essa política no MINIX 3 sem alterar os mecanismos de baixo nível do núcleo (`minix/kernel/proc.c`), a estratégia adotada foi unificar os processos de usuário em uma única fila estática. Modificou-se o servidor de escalonamento (`minix/servers/sched/schedule.c`) e o gerenciador de processos (`minix/servers/pm/utility.c`) para que funções como `do_start_scheduling()`, `do_nice()` e `nice_to_priority()` alocassem todos os processos de usuário exclusivamente na fila `USER_Q` (prioridade 7). Processos de sistema, contudo, mantiveram suas prioridades originais inalteradas.

Com o confinamento em uma fila única, a política dinâmica de múltiplas filas (MLFQ) foi completamente desativada. Removeu-se o rebaixamento de prioridade em `do_noquantum()` (eliminando `rmp->priority += 1`) e o mecanismo de envelhecimento (*aging*), esvaziando as funções `balance_queues()` e `init_scheduling()`, além de remover suas chamadas de atualização de *clock* em `main.c`. Assim, ao expirar o *quantum* ou ao desbloquear de uma operação de E/S, a reinserção do processo na cauda da fila `USER_Q` via `schedule_process_local()` reproduz, de forma natural, o comportamento circular puro do *Round-Robin*.

O diferencial desta implementação é a adoção de um *quantum* dinâmico, definido pela proporção $\frac{1000}{n}$ ms, onde n representa a quantidade de processos de usuário prontos. A contagem n é obtida iterando sobre os processos e utilizando `sys_getinfo(GET_PROC)` em conjunto com a macro `proc_is_runnable()` ([MINIX 3 Project]). Para suportar essa lógica, incluíram-se as bibliotecas `<minix/timers.h>` e `"kernel/proc.h"` e criaram-se as rotinas auxilia-

res `apply_variable_quantum()` e `reschedule_all_user_quantum()`. Desse modo, o tempo de CPU é recalculado e aplicado dinamicamente (por meio de `sys_schedule()`) sempre que um processo é iniciado (`do_start_scheduling()`), encerrado (`do_stop_scheduling()`) ou quando esgota sua fatia de tempo (`do_noquantum()`).

6.3. Múltiplas Filas com Quantum Exponencial

O segundo algoritmo implementado altera a política de múltiplas filas do MINIX 3, substituindo o *quantum* fixo por fatias de tempo crescentes. A modificação baseia-se no sistema CTSS, que operava no IBM 7094 ([Tanenbaum 2016, p. 111]), partindo do pressuposto de que conceder fatias de CPU progressivamente maiores reduz o *overhead* causado pelo excesso de trocas de contexto. Para isso, o projeto define que, ao esgotar seu tempo de CPU alocado, o processo é rebaixado à fila inferior, recebendo um novo *quantum* que cresce em potências de dois. Na última fila, os processos remanescentes são escalonados segundo a política *Round-Robin* com o *quantum* correspondente àquele nível.

Para implementar as alterações propostas, as modificações foram feitas no arquivo `minix/servers/sched/schedule.c`. Vale ressaltar que as novas regras de escalonamento aplicam-se exclusivamente aos processos de usuário, ou seja, aqueles que estão em filas acima de `USER_Q` 7. Para uma implementação coerente, definiu-se uma nova fatia de tempo inicial correspondente a 10% do valor padrão que o MINIX 3.4.0rc6 define para processos de usuário. Isso foi feito a partir da modificação do valor de `DEFAULT_USER_TIME_SLICE` para o valor de 20 no lugar de 200. Isso permitiu verificar nos resultados a influência do crescimento progressivo do tempo de CPU para processos longos.

Com o objetivo de desenvolver essa lógica, a função `user_quantum_for_priority` foi criada para calcular, em potências de dois, o *quantum* correspondente a cada nível de fila. O controle do tempo de CPU atribuído a cada processo, por sua vez, é atualizado pela função `update_user_quantum`. A chamada dessa função ocorre em situações distintas: em `do_start_scheduling()`, para inicializar o processo com o *quantum* base; em `do_noquantum()`, para atualizar o *quantum* no momento da preempção por esgotamento do tempo de CPU do processo; no caso `SCHEDULING_INHERIT`, para assegurar que o filho, criado na mesma fila de prioridade que o pai, tenha seu *quantum* corretamente definido; e, por fim, em `do_nice()`, para que a mudança de prioridade ajuste a fatia de tempo adequada.

Para assegurar o comportamento definido no projeto, o rebalanceamento das filas executado periodicamente em determinados ciclos de *clock* foi desativado. Para isso, em `init_scheduling()` os trechos que fazem a chamada de `sys_setalarm(balance_timeout)` foram comentados, de modo que em `minix/servers/sched/main.c` não haja a invocação de `balance_queues()`.

6.4. First-Come, First-Served (FCFS)

O algoritmo *First-Come, First-Served* atende os processos estritamente na ordem de chegada, sem preempção por tempo. Sua implementação no MINIX 3 exigiu desativar a prioridade dinâmica de múltiplas filas e as interrupções temporais para as tarefas de usuário, garantindo que executem de forma exclusiva até sua finalização ou bloqueio por E/S.

No espaço de núcleo, as modificações foram realizadas no arquivo `/minix/kernel/proc.c`, onde a preempção foi desabilitada na função `proc_no_time()`. A chamada `notify_scheduler(p)`, que efetuava o dequeue do processo ao fim do tempo, foi substituída pela renovação contínua da sua fatia de tempo (`p->p_cpu_time_left`). Essa abordagem mantém a CPU com o processo atual sem quebrar os cálculos estatísticos internos do microkernel com um *quantum* infinito.

No espaço de usuário, as alterações foram realizadas no arquivo `/minix/servers/sched/schedule.c`. A estabilidade do sistema foi assegurada mantendo as filas 0 a 6 inalteradas para os servidores críticos. O FCFS foi aplicado exclusivamente aos processos do usuário. Na função `do_start_scheduling()`, todo processo com prioridade igual ou superior a 7 passou a ser alocado forçadamente em uma única fila (`USER_Q`). Com todos os processos de usuário concentrados nessa fila e sem variação de prioridade, os mecanismos de rebaixamento implementados em `do_noquantum()` e de promoção por *aging* implementados em `balance_queues()` foram completamente removidos. Por fim, para evitar interferências externas, a função `do_nice()` foi modificada para retornar `OK` imediatamente, impedindo a alteração manual de prioridades pelos usuários.

7. Resultados Obtidos e Discussão

Para garantir a robustez estatística de nossos resultados, realizaram-se 5 execuções para cada algoritmo, e calcularam-se a média e o desvio-padrão de cada um deles. Ao final das execuções, foi desenvolvido um gráfico de barras para cada classe de processos, no qual o eixo X representa o número de processos e o eixo Y representa o tempo de execução médio. Esses resultados são apresentados na Figura 7 e na Figura 8. Neles são apresentados em azul o algoritmo padrão do MINIX, em amarelo o FCFS, em verde o *Round-Robin* (RR) e em vermelho o Múltiplas Filas com *Quantum* Exponencial (MF).

Também é válido ressaltar que todos os testes de desempenho foram executados no mesmo computador, o qual foi mantido sem a execução de tarefas paralelas ou uso pessoal, de modo que os resultados obtidos fossem consistentes. A configuração do computador utilizado foi um processador 12th Gen Intel(R) Core(TM) i5-12500H com 16GB de RAM, com o Windows 11 instalado. Por fim, a versão da Oracle VirtualBox é a 7.2.6.

7.1. Resultados para testes de processos *CPU-bound*

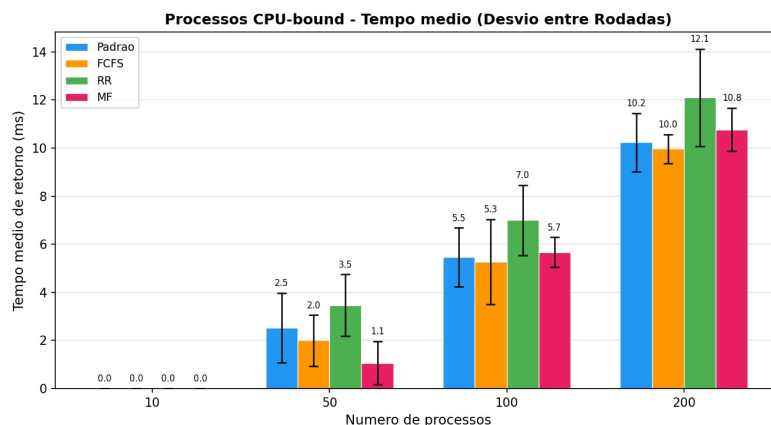


Figura 7. Resultados obtidos para processos *CPU-bound*.

A análise dos resultados para os processos *CPU-bound* (Figura 7) revela que, com uma carga base de 10 processos, todos os algoritmos concluíram as tarefas de forma quase instantânea, registrando 0.0 ms (valor abaixo do limite de medição do sistema). Porém, a partir de 50 processos, o algoritmo RR passou a ter o pior desempenho. Por forçar preempções baseadas puramente no tempo (*quantum*), os processos esgotam sua cota continuamente antes do fim da tarefa, gerando um número excessivo de trocas de contexto que eleva consideravelmente o tempo total de execução.

As divergências entre as políticas ficam nítidas com o aumento da carga. Para 50 processos, o MF destaca-se com o menor tempo médio (1.1 ms), pois compensa as preempções iniciais realocando os processos para filas inferiores com fatias de tempo exponencialmente maiores. No entanto, em cenários de alta concorrência (100 e 200 processos), o MF perde eficiência, superando apenas o RR, porque o imenso *overhead* inicial das trocas de contexto não consegue ser compensado, além de gerar risco de *starvation* devido ao acúmulo de processos nas últimas filas.

Nesses cenários extremos de 200 processos, o FCFS e o algoritmo Padrão do MINIX mostram a melhor escalabilidade. O FCFS atingiu 10.0 ms, resultado teoricamente justificado por sua natureza não preemptiva, que elimina as trocas de contexto e dedica a CPU integralmente à execução útil. De outro modo, o RR sofre o maior impacto sob carga pesada, registrando a pior média (12.1 ms) e a maior variância, consequência direta de um *quantum* particionado ao extremo ao dividir a CPU com muitos processos (ex: $\frac{1000}{200} = 5\text{ ms}$), o que acentua a degradação pelas preempções.

Por fim, para 200 processos, o algoritmo padrão do MINIX apresentou a segunda melhor média (10.2 ms), mas seu desvio-padrão indica que otimizações internas do escalonador dinâmico permitiram picos na casa dos 9.0 ms em rodadas específicas. Contudo, ao considerar a sobreposição das margens de erro, conclui-se que há um empate técnico entre o Padrão do MINIX e o FCFS. A principal diferença reside na estabilidade, pois o FCFS entrega maior previsibilidade e menor variância ao dispensar totalmente as interrupções de prioridade.

7.2. Resultados para testes de processos *IO-bound*

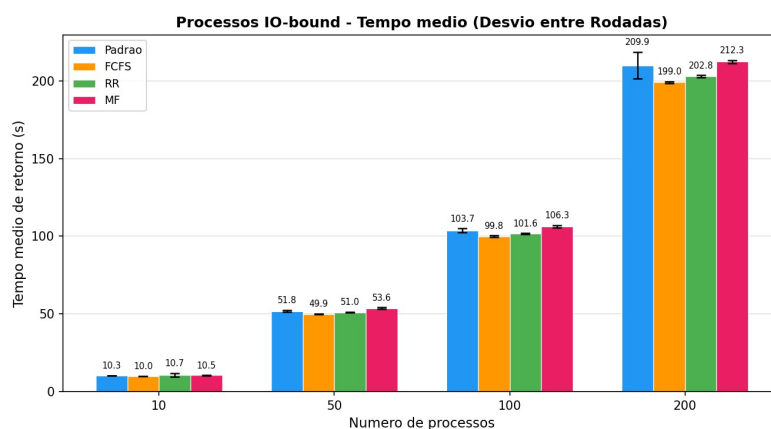


Figura 8. Resultados obtidos para processos *IO-bound*

A análise da Figura 8 revela tempos na ordem de segundos, refletindo a natureza da carga *IO-bound*, que passa a maior parte do tempo bloqueada aguardando a conclusão

de operações de E/S. Diferente do cenário *CPU-bound*, o desvio-padrão foi muito mais uniforme: as barras de erro extremamente curtas indicam um comportamento dos escalonadores previsível e consistente entre as rodadas.

Avaliando os algoritmos, o FCFS apresentou a melhor performance em todas as faixas, com tempo médio variando de 10.0s (10 processos) até 199.0s (200 processos). Embora seja considerado um algoritmo teoricamente ineficiente, esse resultado deve-se à eliminação da sobrecarga (*overhead*). Neste cenário, o Efeito Comboio não ocorre, pois todos os processos realizam I/O e bloqueiam rapidamente. Ao remover os cálculos de envelhecimento e o trânsito dinâmico entre as 16 filas, o sistema apenas reinsere o processo no final da fila única, poupando ciclos do microkernel.

Seguido do FCFS, o RR apresentou o segundo melhor desempenho (202.8s no pior cenário). O tamanho do *quantum* tem pouco impacto neste caso, pois os processos bloqueiam por E/S antes de esgotarem suas fatias de tempo. Quando o processo fica pronto novamente, ele é enviado ao final da fila, tornando seu fluxo muito semelhante ao do FCFS, diferindo apenas pelo leve *overhead* de monitoramento do *timer*.

Por outro lado, algoritmos de prioridades dinâmicas (padrão do MINIX e MF) mostraram-se menos favoráveis para este perfil de carga. O MF registrou o pior tempo (212.3s) com 200 processos. Projetado para beneficiar processos que usam muita CPU através do aumento do *quantum*, o MF não beneficia processos que bloqueiam constantemente, e a manipulação das filas provoca atrasos. O mesmo princípio se aplica ao algoritmo padrão do MINIX (209.9s), onde processos limitados por I/O são rebaixados com menos frequência e acabam congestionando as filas de alta prioridade, gerando excesso de cálculos para balancear uma carga que apenas aguarda os periféricos.

Conclui-se que a aproximação dos tempos de retorno observada entre as diferentes abordagens ocorre porque, em cenários limitados por E/S, o peso de processamento do escalonador perde relevância. Nesses testes, o gargalo dominante do sistema passa a ser a espera pela resposta do hardware periférico, o que nivela os tempos absolutos de retorno, diminuindo o impacto do algoritmo de escalonamento adotado.

8. Conclusão

Este projeto teve como objetivo principal analisar os diferentes comportamentos de algoritmos clássicos de escalonamento e compará-los com o algoritmo padrão do MINIX. Através da coleta e análise dos tempos médios de execução e do desvio-padrão para processos CPU-bound e IO-bound, em diferentes quantidades de processos, foi possível analisar como as características de cada método afetam no desempenho da CPU.

O algoritmo padrão do MINIX 3, por exemplo, mesmo possuindo tempos médios de execução maiores que de outros métodos em certas situações, em algumas execuções ele foi capaz de atingir tempos médios muito menores que seus pares, demonstrando a capacidade do algoritmo padrão do SO. Para o FCFS, foi possível obter resultados satisfatórios, porém, em alguns experimentos, foi possível verificar um alto desvio-padrão, indicando que o algoritmo é bem instável quando comparado ao resto. O Round-Robin com *quantum* variável foi o pior desempenho deste experimento, principalmente em casos com muitos processos, já que, por conta da grande divisão do *quantum*, há uma quantidade excessiva de trocas de contexto. Por fim, o múltiplas filas com *quantum* exponencial

teve resultados mistos, sendo capaz de desempenhar bem em processos CPU-bound, mas em casos de processos IO-bound, possuiu as piores performances.

Concluimos, portanto, que o algoritmo padrão do MINIX ainda se mostra como uma alternativa adequada para um sistema operacional que deve ser escalável, já que ele foi capaz de performar de maneira satisfatória em grande parte dos casos em que testamos, diferente dos outros métodos, sendo capaz, até, de superá-los em algumas execuções.

9. Divisão das tarefas

Tabela 1. Divisão de tarefas entre os membros do grupo

Integrante	Atividades
Breno C. R. Nakamura	Etapa #2: FCFS e primeiros testes; Etapa #3: Seções: Resumo, 1, 6.1, 6.4, 7; Revisão Geral do Documento.
Estevan T. Santos	Execução dos Testes; Plotagem dos Gráficos; Elaboração do Vídeo para Entrega.
Gabriel L. Bonavina	Etapa #2: Round-Robin e primeiros testes; Etapa #3: Seções: 2, 3, 4, 5, 6.2 e 7.
Jonas L. Durão	Etapa #2: Round-Robin e primeiros testes; Etapa #3: Seções: 2, 3, 4, 6.2 e 7.
Natália V. A. do Val	Etapa #2: FCFS e primeiros testes; Etapa #3: Seções: 6.1, 6.4, 7; Revisão Geral do Documento.
Vinícius A. L. Andrade	Etapa #2: Múltiplas Filas com Quantum Exponencial e primeiros testes; Etapa #3: Seções: 6.3, 7 e 8; Revisão Geral do Documento.

Referências

- MINIX 3 Project. Minix 3 wiki documentation. <https://wiki.minix3.org/>. Accessed: 2026-06-04.
- Tanenbaum, A. S. (2016). *Sistemas Operacionais Modernos*. São Paulo: Pearson Education Brasil, 4th edition.
- Tanenbaum, A. S. and Woodhull, A. S. (2008). *Sistemas Operacionais: Projeto e Implementação*. Porto Alegre: Bookman, 3rd edition.